

Linked List

Introduction, Explanation & Interview Preparation Guide

1. What is a Linked List?

A **linked list** is a linear data structure in which elements, called **nodes**, are not stored in contiguous memory locations. Instead, each node stores its own data along with a **reference (pointer)** to the next node in the sequence. The list is accessed starting from a special node called the **head**, and the last node points to **null**, signaling the end of the list.

This design makes linked lists fundamentally different from arrays: instead of relying on index-based contiguous storage, a linked list relies on explicit links between elements. This gives it very different performance trade-offs for insertion, deletion, and access.

Basic Node Structure (C-style pseudocode)

```
struct Node {
    int data;           // value stored in the node
    Node* next;         // pointer/reference to the next node
};

Node* head;           // entry point to the list
```

2. Types of Linked Lists

Type	Description
Singly Linked List	Each node points only to the next node. Traversal is possible in one direction only (forward).
Doubly Linked List	Each node has two pointers: one to the next node and one to the previous node. Allows traversal in both directions.
Circular Linked List	The last node points back to the head instead of null, forming a loop. Can be singly or doubly circular.

3. Linked List vs. Array

Aspect	Array	Linked List
Memory layout	Contiguous	Scattered (linked via pointers)
Access by index	O(1)	O(n)
Insertion/Deletion at start	O(n) (shifting required)	O(1)
Insertion/Deletion at end	O(1) amortized (dynamic array)	O(n) singly / O(1) with tail pointer
Memory overhead	Low (just data)	Higher (data + pointer(s) per node)
Cache performance	Better (locality of reference)	Poorer (nodes scattered in memory)
Dynamic resizing	Costly (reallocation)	Natural / flexible

4. Time Complexity of Common Operations

Operation	Singly Linked List	Doubly Linked List
Access (by index)	$O(n)$	$O(n)$
Search	$O(n)$	$O(n)$
Insert at head	$O(1)$	$O(1)$
Insert at tail (no tail ptr)	$O(n)$	$O(n)$
Insert at tail (with tail ptr)	$O(1)$	$O(1)$
Delete at head	$O(1)$	$O(1)$
Delete at tail	$O(n)$	$O(1)$ (with tail ptr)
Insert/Delete at known node	$O(1)$ (with prev ref)	$O(1)$

5. Core Operations (Python Implementation)

Below is a minimal singly linked list implementation covering the operations most commonly asked about in interviews: insertion, deletion, search, traversal, and reversal.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def insert_at_head(self, data):
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node

    def insert_at_tail(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return
        curr = self.head
        while curr.next:
            curr = curr.next
        curr.next = new_node

    def delete_value(self, key):
        curr = self.head
        prev = None
        while curr and curr.data != key:
            prev = curr
            curr = curr.next
        if not curr:
            return False          # value not found
        if not prev:
            self.head = curr.next # deleting head
        else:
            prev.next = curr.next
        return True

    def search(self, key):
        curr = self.head
        while curr:
            if curr.data == key:
                return True
            curr = curr.next
        return False

    def reverse(self):
        prev = None
        curr = self.head
        while curr:
            nxt = curr.next
            curr.next = prev
            prev = curr
            curr = nxt
```

```
self.head = prev

def print_list(self):
    curr = self.head
    while curr:
        print(curr.data, end=" -> ")
        curr = curr.next
    print("None")
```

6. Important Interview Questions

Below are frequently asked linked list interview questions, grouped by difficulty, with short explanations of the approach expected in interviews.

Conceptual / Basics

Q1. What is a linked list and how does it differ from an array?

- Linked list = nodes connected via pointers, non-contiguous memory, dynamic size.
- Array = contiguous memory, fixed size (unless dynamic array), $O(1)$ random access.

Q2. What are the advantages and disadvantages of linked lists?

- Advantages: dynamic size, $O(1)$ insert/delete at known position, no memory wastage from over-allocation.
- Disadvantages: $O(n)$ access time, extra memory for pointers, poor cache locality.

Q3. Singly vs. Doubly vs. Circular linked list — when would you use each?

- Singly: simplest, least memory, forward traversal only (e.g. stacks, simple queues).
- Doubly: need backward traversal or $O(1)$ deletion given only a node (e.g. LRU cache).
- Circular: round-robin scheduling, continuous looping structures, circular buffers.

Classic Problem-Solving Questions

Q4. How do you detect a cycle in a linked list?

- Use Floyd's Cycle Detection (slow & fast pointer / 'tortoise and hare').
- Move slow by 1 step and fast by 2 steps; if they meet, a cycle exists.
- Time: $O(n)$, Space: $O(1)$.

Q5. How do you find the starting node of a cycle?

- After slow and fast meet, reset one pointer to head.
- Move both pointers one step at a time; they meet exactly at the cycle's start node.

Q6. How do you reverse a linked list (iterative and recursive)?

- Iterative: maintain prev, curr, next pointers and re-link one node at a time — $O(n)$ time, $O(1)$ space.
- Recursive: reverse the rest of the list first, then fix the current node's links — $O(n)$ time, $O(n)$ space (call stack).

Q7. How do you find the middle node of a linked list in one pass?

- Use slow/fast pointers: slow moves 1 step, fast moves 2 steps.
- When fast reaches the end, slow is at the middle.

Q8. How do you detect and remove duplicates from a linked list?

- With extra space: use a hash set to track seen values while traversing — $O(n)$ time.
- Without extra space: use two pointers (current and runner) — $O(n^2)$ time, $O(1)$ space.

Q9. How do you merge two sorted linked lists?

- Use a dummy head node; compare current nodes of both lists and attach the smaller one.
- Advance the pointer of the list whose node was attached; repeat until one list is exhausted, then attach the remainder.

- Time: $O(n + m)$.

Q10. How do you find the Nth node from the end of a linked list?

- Use two pointers: advance the first pointer N nodes ahead.
- Then move both pointers together until the first reaches the end; the second is now at the Nth node from the end.
- Single pass, $O(n)$ time, $O(1)$ space.

Q11. How do you check if a linked list is a palindrome?

- Find the middle using slow/fast pointers, reverse the second half, then compare both halves.
- Optionally restore the list by reversing the second half back.
- Time: $O(n)$, Space: $O(1)$.

Q12. How do you remove the Nth node from the end in one pass?

- Use a dummy node before head, and two pointers separated by $N+1$ nodes.
- Move both until the fast pointer reaches the end, then unlink the target node.

Q13. How do you add two numbers represented as linked lists (digits stored in reverse order)?

- Traverse both lists simultaneously, adding corresponding digits plus carry.
- Create new nodes for the result list; handle any leftover carry at the end.

Q14. How would you flatten a multilevel linked list (with child pointers)?

- Do a depth-first traversal: whenever a node has a child, recursively flatten the child list, then splice it between the current node and its next node.

Q15. How do you sort a linked list?

- Merge sort is preferred: $O(n \log n)$ time, and works well since linked lists don't need random access (unlike quicksort, which relies on it for good performance).
- Split the list into halves using slow/fast pointers, recursively sort, then merge.

Design / Applied Questions

Q16. How would you implement an LRU cache using a linked list?

- Use a doubly linked list + hash map: hash map gives $O(1)$ node lookup, doubly linked list gives $O(1)$ removal/reinsertion to track recency order.
- Most recently used node moves to the front (or back); least recently used is evicted from the other end.

Q17. Why is a doubly linked list preferred over a singly linked list for implementing an LRU Cache?

- Deleting a node in $O(1)$ requires access to the previous node.
- A singly linked list only knows the next pointer, so removing a specific node needs $O(n)$ traversal to find its predecessor; a doubly linked list avoids this.

Q18. Array-based list vs. linked list: which would you choose for a text editor's undo feature, and why?

- A doubly linked list is often preferred since undo/redo needs $O(1)$ insertion and traversal in both directions without shifting elements, unlike an array.

Q19. How is a linked list used to implement a stack or a queue?

- Stack: push/pop at head — both $O(1)$.
- Queue: enqueue at tail, dequeue at head — $O(1)$ each if a tail pointer is maintained.

Q20. What is a skip list, and how does it relate to linked lists?

- A skip list is a linked list augmented with multiple layers of 'express lane' pointers, allowing average $O(\log n)$ search, insertion, and deletion — used in some databases and Redis.

7. Quick Interview Tips

- Always clarify whether the list is singly, doubly, or circular before answering.
- Draw the list on paper/whiteboard and trace pointer changes step by step — most linked list bugs come from mis-ordering pointer updates.
- The **slow/fast (two-pointer) pattern** solves a huge fraction of linked list problems: cycle detection, middle node, Nth-from-end, palindrome check.
- Use a **dummy/sentinel head node** when the head itself might change (deletion, merging, reversal) — it avoids special-casing the head.
- State time and space complexity explicitly after solving — interviewers almost always ask for it.
- Practice both iterative and recursive solutions for reversal; recursive uses $O(n)$ stack space, which interviewers may probe.

Prepared as a quick-reference guide for linked list concepts and coding interview practice.